PostgreSQL

Ecosistema e fondamenti

Giorno 1, pomeriggio

Marco De Giovanni · 20 ottobre 2026

AGENDA INTRODUZIONE

Struttura del pomeriggio

01

Ciclo 3 · Teoria

Indici B-Tree, EXPLAIN, pg_stat_statements

→

02

Laboratorio · Fase 3

il sito rallenta (indici, EXPLAIN, pg_stat_statements)

→

03

Ciclo 4 · Teoria

Array, UUID, JSONB, trigger, GIN e BRIN, VACUUM

→

04

Laboratorio · Fase 4

marketing e manutenzione (JSONB, GIN/BRIN, VACUUM), debriefing

01

CICLO 3 · TEORIA

Indici e piani

Indici B-Tree, `EXPLAIN`, `pg_stat_statements`

3.01 CICLO 3 · INDICI E PIANI

Il pomeriggio: perché questa query è lenta?

+ Il caso della mattina, un anno dopo: il sito è in produzione con 1.000.000 di ordini, con **solo** l'indice della primary key su `orders`.

IL PERCORSO

Indici → `EXPLAIN` → `pg_stat_statements` → Tipi avanzati e JSONB → GIN / BRIN → `VACUUM`

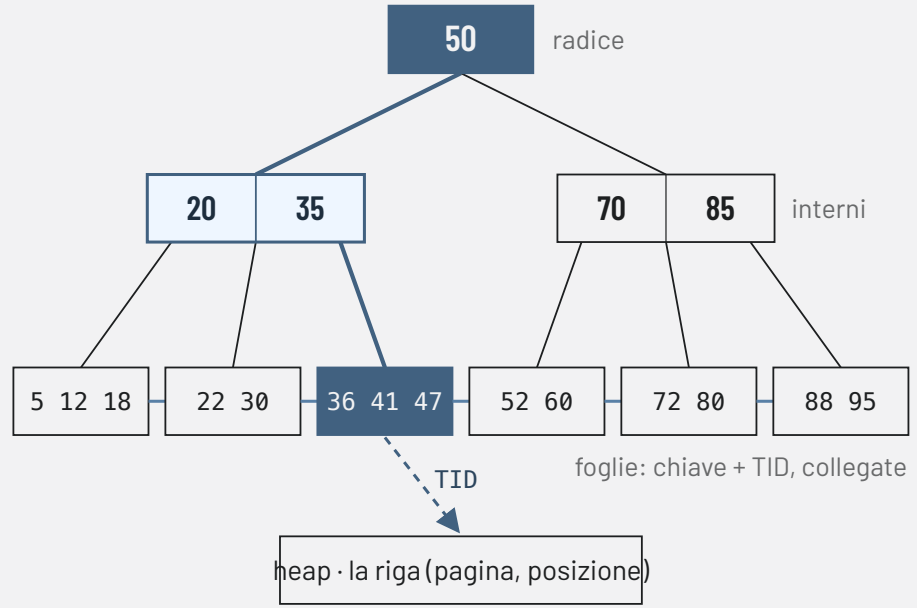
+ Metodo per tutto il pomeriggio: **misura → intervieni → rimisura**.

Indice B-Tree: com'è fatto

- È l'indice di default (`CREATE INDEX`): un albero bilanciato di pagine da 8 kB, 3–4 livelli anche con centinaia di milioni di righe.
- PostgreSQL usa una variante **B+Tree** (algoritmo di Lehman-Yao):
  - nodi interni: solo chiavi di instradamento;
  - **foglie**: tutte le chiavi, ciascuna con il TID (*tuple identifier* = `ctid`: pagina e posizione della riga);
  - le pagine dello stesso livello sono **collegate** → gli intervalli scorrono lateralmente.
- Accelera `=`, `IN`, `<`, `>`, `BETWEEN`, `ORDER BY … LIMIT`, `IS NULL`, `LIKE 'abc%'` (solo con collation `C` — non basta `C.UTF-8` — oppure con `text_pattern_ops`).
- `PRIMARY KEY` e `UNIQUE` creano un B-Tree; **una foreign key no**.

```sql
CREATE INDEX idx_orders_customer ON orders (customer_id);
-- equivalente esplicito
CREATE INDEX ON orders USING btree (customer_id);
-- per LIKE 'abc%' con collation non C
CREATE INDEX ON customers (email text_pattern_ops);
```

B+TREE · RICERCA DI `id = 41`



3 pagine d'indice + 1 di heap

Selettività: indice, bitmap o scansione

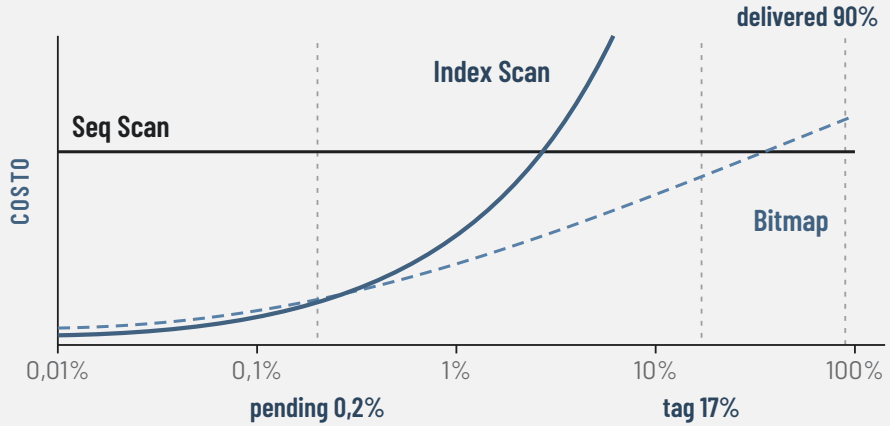
STRADA	COSTO	QUANDO VINCE
`Seq Scan`	legge tutte le pagine in sequenza: dipende dalla dimensione della tabella	molte righe (es. `delivered`, ~90%)
`Index Scan`	un salto casuale nell'heap per ogni riga trovata	pochissime righe (es. `pending`, ~0,2%)
`Bitmap Scan`	raccoglie i riferimenti, poi legge ogni pagina una volta in ordine fisico	via di mezzo

- Il planner sceglie la strada **più economica** in base alle statistiche (`ANALYZE`).
- "L'indice esiste" non significa "l'indice viene usato": un indice su una colonna poco selettiva (flag, stato dominante) è quasi sempre inutile.
- Le soglie dipendono da dimensione della tabella, distribuzione fisica e `random_page_cost`.

SQL

```sql
CREATE INDEX idx_orders_status ON orders (status);
ANALYZE orders;
-- poche righe: Index/Bitmap Scan
EXPLAIN SELECT * FROM orders WHERE status = 'pending';
-- molte righe: Seq Scan
EXPLAIN SELECT * FROM orders WHERE status = 'delivered';
```

COSTO IN FUNZIONE DELLA % DI RIGHE



Asse orizzontale in scala logaritmica

Indici compositi e parziali

Composito `(customer_id, created_at)` — regola del **prefisso sinistro**

QUERY	USA L'INDICE?
`WHERE customer_id = 1`	sì
`WHERE customer_id = 1 ORDER BY created_at DESC LIMIT 3`	sì, **senza** `Sort`
`WHERE created_at > …`	di norma no (PG 18: *skip scan* solo se la prima colonna ha pochi valori distinti)

+ Prima le uguaglianze, poi intervallo/ordinamento; un composito `(a, b)` rende inutile un indice su `(a)`.

SQL

```sql
CREATE INDEX idx_orders_cust_date
  ON orders (customer_id, created_at DESC);
```

Parziale … `(created_at) WHERE status = 'pending'`

+ Indicizza solo le righe che servono: ~64 kB invece di ~30 MB.
+ Usato solo se la query **implica** il predicato (attenzione agli statement parametrizzati).
+ Usi: righe attive, soft delete (`deleted_at IS NULL`), unicità condizionale.

SQL

```sql
CREATE INDEX idx_orders_pending
  ON orders (created_at)
  WHERE status = 'pending';

CREATE UNIQUE INDEX ON users (email)
  WHERE deleted_at IS NULL;
```

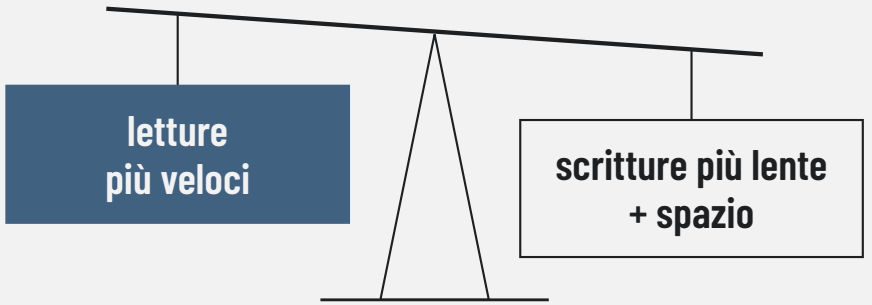
Il costo degli indici

- Ogni indice rallenta `INSERT`, `UPDATE`, `DELETE` e occupa spazio.
- Un `UPDATE` su una colonna con indice B-Tree impedisce gli **HOT update** (e richiede `VACUUM` degli indici); non vale per i BRIN (da PG 16).
- Indici mai usati: `pg_stat_user_indexes.idx_scan = 0`.
- In produzione: `CREATE INDEX CONCURRENTLY` (non blocca le scritture; fuori transazione; se fallisce lascia un indice `INVALID`), `REINDEX … CONCURRENTLY`.
- **Indicizza le colonne FK** usate per join o cancellazioni sul padre.

SQL

```sql
CREATE INDEX CONCURRENTLY idx_orders_customer
  ON orders (customer_id);
SELECT indexrelname, idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid))
FROM pg_stat_user_indexes WHERE relname = 'orders';
DROP INDEX CONCURRENTLY idx_orders_status;
```

IL COMPROMESSO DI OGNI INDICE



Algoritmi di join

ALGORITMO	COME LAVORA	VA BENE QUANDO
`Nested Loop`	per ogni riga esterna cerca nell'interna	poche righe esterne + indice sull'interna
`Hash Join`	hash table della tabella più piccola, poi probe	tabelle grandi; `Batches > 1` = non stava in `work_mem`
`Merge Join`	fonde due input ordinati	input già ordinati (indici)

+ `NOT EXISTS` → anti-join efficiente (`Hash Right Anti Join` da PG 16; prima `Hash Anti Join`).
+ `NOT IN (sottoquery)` **non** diventa anti-join: su 1M di righe un costo dell'ordine delle centinaia di milioni (circa 770 milioni nella nostra prova) → **mai** con `ANALYZE`.
+ `enable_hashjoin`, `enable_nestloop`... solo per diagnosi.

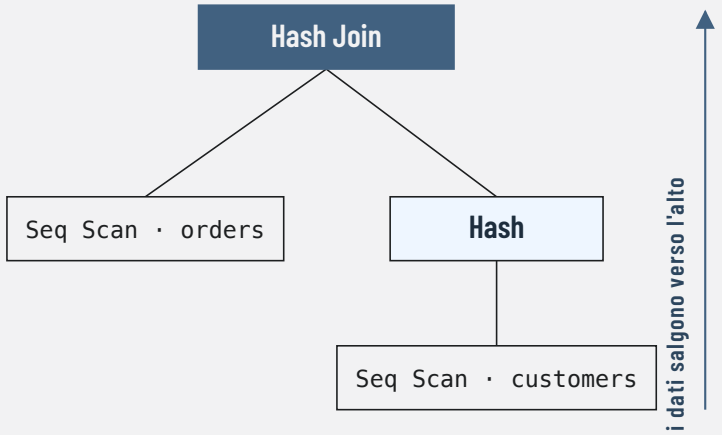
Cos'è un piano di esecuzione

- Il planner trasforma la query in un **albero di nodi**; si legge **dal nodo più indentato verso l'alto**.
- `EXPLAIN` → piano **stimato**, la query non viene eseguita.
- `EXPLAIN ANALYZE` → la query **viene eseguita** (con DML: in transazione e `ROLLBACK`).
- Opzioni: `BUFFERS` (automatico con `ANALYZE` da PG 18), `VERBOSE`, `SETTINGS`, `FORMAT JSON`.
- `cost=0.29..8.31`: avvio..totale in **unità arbitrarie** (non millisecondi), da `seq_page_cost` 1.0, `random_page_cost` 4.0 (su SSD spesso 1.1), `cpu_tuple_cost` 0.01.
- In DBeaver: `EXPLAIN (ANALYZE, BUFFERS) …` eseguito come una query; vista grafica con `Ctrl+Shift+E` (macOS `^⇧E`).

SQL · LA QUERY DI QUESTO PIANO

```sql
EXPLAIN SELECT o.id, o.total, c.name
FROM orders o
JOIN customers c ON c.id = o.customer_id;
```

UN PIANO È UN ALBERO



Leggere un nodo: stima contro realtà

RISULTATO · IL PIANO

```
1 Bitmap Heap Scan on orders
2   (cost=4.51..47.76 rows=11 width=84)
3   (actual time=0.02..0.04 rows=9 loops=1)
4   Recheck Cond: (customer_id = 4242)
    Heap Blocks: exact=9
5   Buffers: shared hit=15
6   ->  Bitmap Index Scan on idx_orders_customer
          (cost=0.00..4.51 rows=11 width=0)
          (actual time=0.01..0.01 rows=9 loops=1)
          Buffers: shared hit=6
```

LE SEZIONI DEL NODO

1. **NODO** `Bitmap Heap Scan on orders`
   Operazione e tabella. Il rientro con `->` indica i figli: si eseguono **prima** e passano le righe al padre.

2. **STIMA** `cost · rows · width`
   Previsione del planner: costo d'avvio..totale (unità arbitrarie), righe e byte per riga. Il costo si confronta solo tra piani **della stessa query**.

3. **MISURA** `actual time · rows · loops`
   Solo con `ANALYZE`: ms alla prima e all'ultima riga, righe reali, quante volte il nodo è stato eseguito.

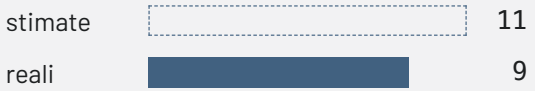
4. **DETTAGLI** `Recheck Cond · Heap Blocks`
   La condizione applicata e le pagine di tabella visitate (`exact`: la bitmap conosce la riga esatta).

5. **BUFFERS** `shared hit · read`
   Pagine da cache (`hit`) o da disco (`read`). Il valore è **cumulativo**: comprende i figli.

6. **FIGLIO** `Bitmap Index Scan`
   Legge l'indice e costruisce la bitmap che il nodo padre usa per leggere la tabella.

Leggere un nodo: come confrontare i valori

1 STIMA CONTRO REALTÀ



1,2× di scarto: stima affidabile

Oltre **~10×**, in più o in meno, il planner sceglie con dati sbagliati: verifica le statistiche (`ANALYZE`, slide 3.09).

2 MOLTIPLICA PER LOOPS

```
rows=9 × loops=1          9 righe
rows=9 × loops=1000  9.000 righe
0,04 ms × 1000           40 ms
```

`rows` e `actual time` sono **medie per esecuzione**: un nodo veloce ripetuto mille volte (figlio di un `Nested Loop`) può essere il collo di bottiglia.

3 BUFFERS CUMULATIVI



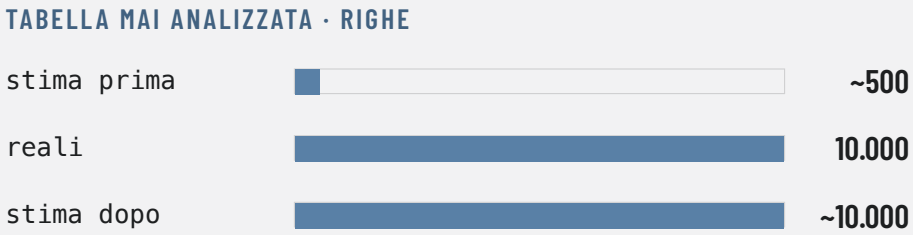
15 – 6 del figlio = **9 pagine di tabella**, come `Heap Blocks: exact=9`. `hit` = cache, `read` = disco: a freddo prevalgono i read, a caldo gli hit.

Cerca anche **`Rows Removed by Filter`** (righe lette e scartate: candidato a un indice) e **`Index Searches`** (PG 18: quante discese nell'indice).

+ Misura **più volte**: prima esecuzione a freddo, poi a caldo.

Statistiche e stime

- `ANALYZE` alimenta `pg_stats`: valori distinti (`n_distinct`: positivo = numero, negativo = frazione), valori più frequenti, istogramma, `correlation`, frazione di `NULL`.
- Stime sbagliate tipiche:
  - statistiche **vecchie** (dopo un carico massivo);
  - colonne **correlate** (il planner assume indipendenza) → `CREATE STATISTICS`;
  - **espressioni** senza statistiche (`attrs->>'x'`).
- Esempio: tabella mai analizzata (con l'autovacuum disattivato) → stima ~500 righe, reali 10.000; dopo `ANALYZE` ~10.000.
- `default_statistics_target` = 100; l'autovacuum esegue anche l'auto-analyze.



Metodi di accesso

DA POCHE RIGHE A TUTTA LA TABELLA

`Index Only Scan` → `Index Scan` → `Bitmap Scan` → `Seq Scan` → `Parallel Seq Scan`

NODO	QUANDO
`Seq Scan`	molte righe o tabella piccola
`Index Scan`	poche righe
`Index Only Scan`	l'indice contiene tutte le colonne + pagine "tutte visibili" (`Heap Fetches` ≈ 0 dopo `VACUUM`); su una tabella appena caricata, senza `VACUUM`, il planner non lo sceglie
`Bitmap Index` + `Bitmap Heap Scan`	numero medio di righe; `exact` o `lossy`; più indici con `BitmapAnd`/`BitmapOr`
`Parallel Seq Scan` sotto `Gather`	tabelle grandi; confronta `Workers Planned` e `Launched`

+ Indici *covering*: `CREATE INDEX … INCLUDE (col)`.

DML e il costo nascosto delle FK

- `EXPLAIN ANALYZE` su `UPDATE`/`DELETE` modifica i dati → `BEGIN; … ROLLBACK;`.
- I vincoli di FK sono **trigger interni**: il piano mostra `Trigger for constraint …: time=….`
- Una foreign key **di default non ha indici**: PostgreSQL indicizza solo la colonna referenziata (PK o `UNIQUE`), quella che referenzia va indicizzata a mano.
- Cancellare un cliente cerca i suoi ordini:

INDICE SU ORDERS.CUSTOMER_ID	TEMPO DEL TRIGGER
presente	frazioni di ms
assente	decine o centinaia di ms (scansione di tutta la tabella)

IN TRANSAZIONE: IL PIANO MOSTRA IL TRIGGER

```
BEGIN;                                                              □ ok
EXPLAIN ANALYZE DELETE FROM customers WHERE id = 42;                □ eseguita
Trigger for constraint orders_customer_id_fkey: time=…              ■ trigger FK
ROLLBACK;                                                           ■ annullata
```

Metodo di tuning

01 Trova la query che **pesa** (`pg_stat_statements`).

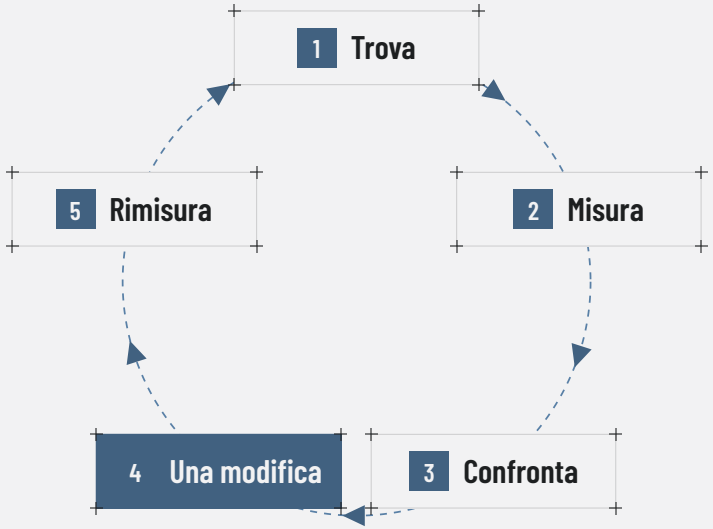
02 Misura con `EXPLAIN (ANALYZE, BUFFERS)`, a caldo, più volte.

03 Confronta stime e realtà; cerca i sintomi: `Rows Removed by Filter`, `Sort … Disk`, `Batches > 1`, `Heap Fetches` alti.

04 **Una** modifica: indice, `ANALYZE`, riscrittura, parametro.

05 Rimisura e confronta.

Strumenti: visualizzatori di piani (attenzione ai dati sensibili), `auto_explain`, `EXPLAIN (GENERIC_PLAN)` (PG 16+) per le query con `$1`.



`pg_stat_statements`: perché e come

- `EXPLAIN` analizza una query nota; **`pg_stat_statements`** dice **quale** query pesa.
- Contatori cumulativi per query **normalizzata** (`WHERE id = 1` e `= 2` → `WHERE id = $1`).
- Richiede `shared_preload_libraries` → **riavvio** (senza, la vista dà errore; `CREATE EXTENSION` funziona comunque):
- `ALTER SYSTEM` sostituisce **l'intero** valore del parametro. Le opzioni `-c` all'avvio vincono su `ALTER SYSTEM`.
- Le statistiche sono raccolte per **tutto il server**; `CREATE EXTENSION` va eseguito in **ogni database** in cui serve la vista.

SQL

```sql
ALTER SYSTEM SET shared_preload_libraries = 'pg_stat_statements';
-- riavvio del server (es. docker compose restart db), poi riconnessione
CREATE EXTENSION pg_stat_statements;
```

`pg_stat_statements`: un esempio

SQL

```sql
SELECT calls, round(total_exec_time) AS total_ms,
       round(mean_exec_time, 2) AS mean_ms, left(query, 52) AS query
FROM pg_stat_statements
ORDER BY total_exec_time DESC LIMIT 4;
```

OUTPUT · VALORI INDICATIVI

```
 calls | total_ms | mean_ms |                       query
-------+----------+---------+-------------------------------------------------
   480 |    21604 |   45.01 | SELECT c.name, count(*), sum(o.total) FROM orders o
 61250 |     5390 |    0.09 | SELECT * FROM orders WHERE customer_id = $1
  2400 |     1872 |    0.78 | SELECT * FROM orders WHERE status = $1 ORDER BY crea
  1200 |      516 |    0.43 | UPDATE orders SET status = $1 WHERE id = $2
(4 rows)
```

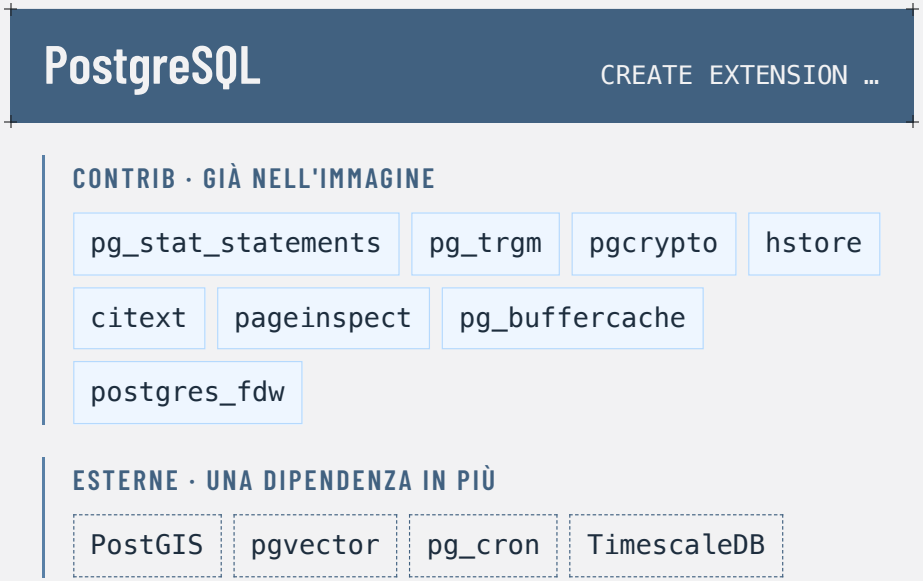
Leggere la vista

- Tre classifiche, tre domande: per **tempo totale** (cosa pesa sul sistema), per **tempo medio** (cosa è lento), per **chiamate** (cosa è frequente).
- Con il carico del laboratorio: il report con `JOIN` domina tempo totale e medio, la ricerca per cliente le chiamate (le classifiche **non coincidono**).
- `pg_stat_statements_reset()` prima di una misura; i contatori sono cumulativi.

COLONNA	SIGNIFICATO
`calls`	esecuzioni
`total_exec_time`, **`mean_exec_time`**	tempo totale e medio (ms)
`rows`	righe restituite
`shared_blks_hit` / **`read`**	I/O
`temp_blks_written`	spill su disco: `work_mem` insufficiente

Le estensioni

- `CREATE EXTENSION`; `pg_extension` (installate), `pg_available_extensions` (disponibili).
- Già nell'immagine (*contrib*): `pg_stat_statements`, `pg_trgm` (`LIKE '%x%'` indicizzabile), `pgcrypto`, `hstore`, `citext`, `pageinspect`, `pg_buffercache`, `postgres_fdw`.
- Esterne: **PostGIS** (geospaziale), `pgvector`, `pg_cron`, TimescaleDB — una dipendenza in più per aggiornamenti e backup.



02

CICLO 3 · PRATICA

Laboratorio

Fase 3 · Il sito rallenta

F3 · 1/2 LABORATORIO · CICLO 3

Fase 3 · Il sito rallenta

Un anno dopo: ShopIT ha **1 milione di ordini** e arrivano segnalazioni di lentezza. Il tuo compito: **diagnosticare e risolvere**, una segnalazione alla volta, con il metodo *misura* → *intervieni* → *rimisura*.

Setup: `sql/bulk_pomeriggio.sql` (schema `public`, ~30 s); `SET max_parallel_workers_per_gather = 0;`.

01

"I miei ordini" è lenta

ordini di un cliente: indice su `customer_id`

02

La dashboard del magazzino è lenta

10 `pending` più vecchi: indice pieno contro **parziale**, confronta tempo, buffer e `pg_relation_size`

03

Il profilo cliente carica lento

ultimi 3 ordini: prova `(created_at, customer_id)` poi `(customer_id, created_at DESC)`, il `Sort` sparisce?

04

"Ho l'indice su status ma è lento"

`delivered` contro `pending`; forza l'indice con `enable_seqscan = off`: migliora o peggiora?

05

Cancellare un cliente è lento

foreign key senza indice (in transazione)

F3 · 2/2 LABORATORIO · CICLO 3

Fase 3 · Quale query pesa di più?

Poi: abilita `pg_stat_statements`, lancia il carico con `pgbench` e leggi le tre classifiche: **quale segnalazione risolvi per prima, e perché?**

DOPO LE SEGNALAZIONI

`pg_stat_statements` → carico con `pgbench` → tre classifiche → **quale risolvi per prima?**

Extra: stima sbagliata su una tabella mai analizzata; `Hash Join` e `work_mem`.

Testo completo e risultati attesi: **dispensa di laboratorio, Fase 3**.

03

CICLO 4 · TEORIA

JSONB e manutenzione

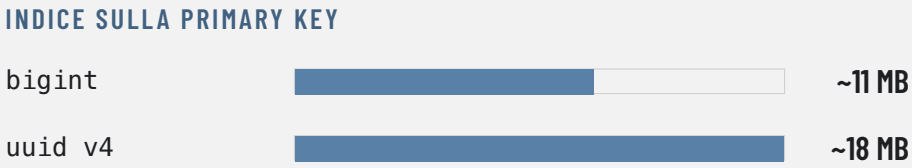
Array, UUID, JSONB, trigger, GIN e BRIN, VACUUM

Array

- Colonna `tags text[]`: `ARRAY['gift','express']` o `'{gift,express}'`; indici da **1**.
- Operatori: `@>` (contiene), `<@`, `&&` (in comune), `= ANY(a)`, `||`; funzioni `unnest`, `array_agg`, `cardinality`.
- Con `||` il letterale va tipizzato: `tags || 'x'::text` (oppure `array_append(tags, 'x')`); `tags || 'x'` dà errore (*malformed array literal*).
- `array_length('{}', 1)` è `NULL`, `cardinality` dà 0.
- Sì per tag e liste piccole; **no** quando servono FK, join o modifiche frequenti a singoli elementi.
- `@>` è indicizzabile con GIN, `= ANY` no.

UUID

- `uuid`: 16 byte (il testo occupa 40). `gen_random_uuid()` (v4 casuale, PG 13+), `uuidv7()` (ordinato nel tempo, PG 18).
- UUID **casuale** come PK: inserimenti sparsi nell'indice → più page split, indice più grande (misurato: ~18 MB contro ~11 MB per `bigint`, inserimento ~3 volte più lento).
- UUID **v7**: prefisso temporale, inserimenti "in coda" come una sequenza.
- `bigint` identity è più piccolo e veloce; UUID quando l'id va generato fuori dal database.



hstore e altri tipi

- **hstore** (contrib): coppie chiave/valore di sole stringhe; storico, oggi si usa `jsonb`.
- Range (`tstzrange`, `daterange`) con vincolo di esclusione: `EXCLUDE USING gist (room WITH =, during WITH &&)` vieta prenotazioni sovrapposte (`btree_gist`).
- `inet`/`cidr`, `interval`, `bytea`, `citext`, `ENUM`.

`json` contro `jsonb`; accesso ai campi

	JSON	JSONB
Memorizza	testo identico all'input	binario decomposto
Spazi, ordine chiavi, duplicati	conservati	no (vince l'ultima chiave)
Interrogazione	rifà il parsing	veloce, indicizzabile

OPERATORE	RESTITUISCE	ESEMPIO
`->`	`jsonb`	`attrs -> 'channel'` → `"app"`
`->>`	`text`	`attrs ->> 'channel'` → `app`
`#>`, `#>>`	per percorso	`attrs #>> '{a,b}'`
`[ ]` **(PG 14+)**	`jsonb`	`attrs['channel']`

+ Usa `jsonb`. Un aggiornamento riscrive l'intero documento.

Operatori JSONB e jsonpath

OPERATORE	SIGNIFICATO
`@>`	contiene il frammento (indicizzabile con GIN)
`?`, `?|`, `?&`	esistenza di chiavi
`||`, `-`, `jsonb_set`	fonde, rimuove, sostituisce
`jsonb_build_object`, `jsonb_agg`	costruisce JSON da righe
`@?`, `@@`	jsonpath: `attrs @@ '$.channel == "app"'`

+ In JDBC `?` è il segnaposto: si scrive `??` o `jsonb_exists(attrs, 'k')`.
+ PG 17: `JSON_VALUE`, `JSON_QUERY`, `JSON_EXISTS`, `JSON_TABLE`.

Quando usare JSONB

SÌ	NO, MEGLIO COLONNE
attributi variabili o sparsi	dati core a struttura stabile
payload di eventi e API	vincoli (FK, `UNIQUE`, `NOT NULL`)
configurazioni	campi filtrati, joinati, aggregati spesso
dati letti per intero	campi aggiornati spesso

+ Il planner **non ha statistiche** sui campi interni: `attrs->>'channel' = 'app'` stimato 5.000 righe, reali 333.334.
+ Rimedio: **indice su espressione** (porta le statistiche) o **colonna generata**. Approccio ibrido: i campi "caldi" diventano colonne.

Trigger e PL/pgSQL

```sql
CREATE FUNCTION set_updated_at() RETURNS trigger AS $$BEGIN NEW.updated_at := now(); RETURN NEW; END$$
LANGUAGE plpgsql;
CREATE TRIGGER trg BEFORE UPDATE ON t FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

- PL/pgSQL: variabili, `IF`, cicli, eccezioni; funzioni, procedure (`CALL`), blocchi `DO`.
- Pro: valgono per ogni client, niente round-trip, utili per audit e invarianti.
- Contro: logica **nascosta**, difficili da testare, lenti sulle scritture massive (per riga).
- Per valori derivati semplici: `DEFAULT`, colonna generata o applicazione.

Oltre il B-Tree: i tipi di indice

INDICE	PER
B-Tree	scalari, ordinamento
GIN	array, `jsonb`, full-text, `LIKE '%x%'` con `pg_trgm`
BRIN	tabelle enormi correlate all'ordine fisico
GiST / SP-GiST	geometrie, range, prossimità, IP
Hash	solo `=` (raro)

GIN: l'indice invertito

- **GIN** = indice **invertito**: per ogni elemento, la lista delle righe che lo contengono.
- Operatori: `@>`, `&&`, `<@`, `=` su array; `@>`, `?`, `?|`, `?&`, `@?`, `@@` su `jsonb` (`jsonb_ops`, default) oppure solo `@>`, `@?`, `@@` (`jsonb_path_ops`, più piccolo).
- Costi: scritture più lente (*pending list*, `fastupdate`), indice grande.
- Con **valori frequenti** il vantaggio crolla: il bitmap finisce per leggere tutte le pagine.



BRIN

- Per ogni gruppo di pagine (default **128**) salva solo **minimo e massimo**.
- Minuscolo: ~24 kB contro ~21 MB di un B-Tree su `created_at`.
- Funziona solo se l'ordine fisico è **correlato** ai valori (`pg_stats.correlation` vicino a ±1): serie storiche in sola aggiunta.
- **Lossy**: indica intervalli di pagine da riverificare (`Recheck`).
- Peggiora con `UPDATE` e inserimenti fuori ordine; parametri `pages_per_range`, `autosummarize`.

BRIN SU CREATED_AT · 128 PAGINE PER INTERVALLO

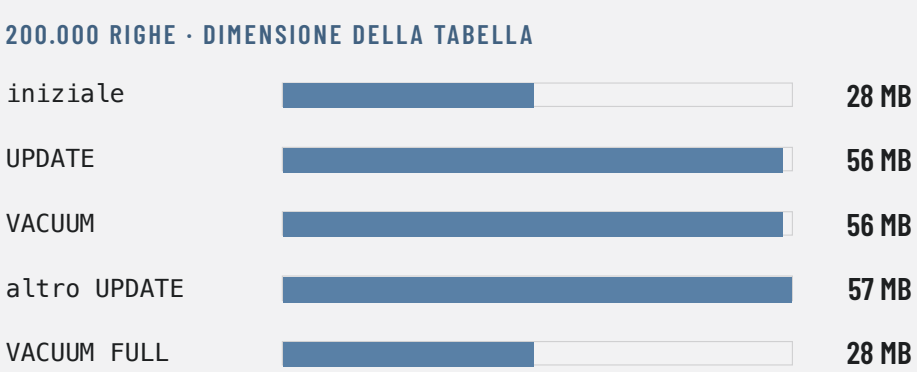
`WHERE created_at = '2026-03-14'`

PAGINE 0-127	PAGINE 128-255	PAGINE 256-383	PAGINE 384-511	PAGINE 512-639	PAGINE 640-767
min 2026-01-01	min 2026-02-01	min 2026-03-01	min 2026-04-01	min 2026-05-01	min 2026-06-01
max 2026-01-31	max 2026-02-28	max 2026-03-31	max 2026-04-30	max 2026-05-31	max 2026-06-30
saltato	saltato	letto · Recheck	saltato	saltato	saltato

Dead tuples, VACUUM e VACUUM FULL

	VACUUM	VACUUM FULL
Cosa fa	rende **riutilizzabile** lo spazio dei dead tuples	**riscrive** la tabella
Dimensione del file	di norma invariata	si riduce
Lock	non blocca letture e scritture	`ACCESS EXCLUSIVE`
Uso	routine (autovacuum)	eccezionale (o `pg_repack`)

+ `VACUUM` aggiorna anche la *visibility map* (Index Only Scan).



Transazioni lunghe: il nemico del `VACUUM`

- `VACUUM` elimina una versione solo se **nessuna transazione** può ancora vederla.
- Una transazione vecchia blocca l'orizzonte di pulizia: `VACUUM VERBOSE` riporta `… are dead but not yet removable`.
- Cause: `idle in transaction`, query lunghe, slot di replica fermi, transazioni preparate.
- Diagnosi (da **un'altra** sessione): `pg_stat_activity` con `backend_xmin` valorizzato.
- Prevenzione: transazioni brevi, `idle_in_transaction_session_timeout`.

Autovacuum

- Scatta quando i dead tuples superano `50 + 0,2 × righe` (analyze: `50 + 0,1 × righe`); da PG 13 anche gli inserimenti.
- Tabelle grandi: fino a PG 17, su 1 miliardo di righe si aspettano 200 milioni di dead tuples; da PG 18 il tetto `autovacuum_vacuum_max_threshold` (100 milioni) lo dimezza.
- Rimedio: parametri **per tabella**:

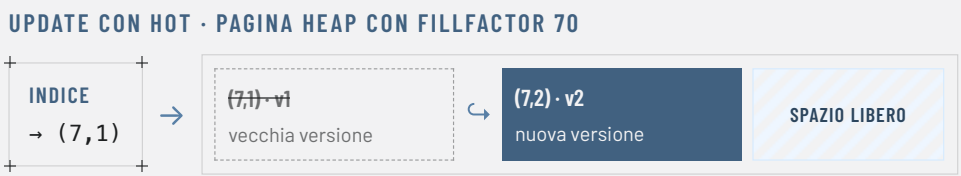
```sql
ALTER TABLE big SET (autovacuum_vacuum_scale_factor = 0.01, autovacuum_vacuum_threshold = 100);
```

- Lavoro rallentato apposta (`cost_delay`, `cost_limit`); `autovacuum_naptime` (1 min).
- Monitoraggio: `pg_stat_user_tables`, `log_autovacuum_min_duration`, `pg_stat_progress_vacuum`. **Mai spegnerlo.**

HOT update e wraparound

- **HOT** (*Heap-Only Tuple*): se l'`UPDATE` non tocca colonne con indice B-Tree (i BRIN non contano, da PG 16) e c'è spazio nella stessa pagina, gli indici non si aggiornano.
  - `fillfactor` < 100 lascia spazio: misurato `fillfactor = 70` → 33.333 HOT su 33.333; `100` → 0.

- **Wraparound**: gli id di transazione sono a 32 bit; `VACUUM` "congela" le righe vecchie. Se non lo fa, PG smette di accettare scritture (restano possibili le letture).
  - monitoraggio: `age(datfrozenxid)`; `autovacuum_freeze_max_age` = 200 milioni.



L'indice non cambia: punta ancora a v1, che rimanda a v2 nella stessa pagina.

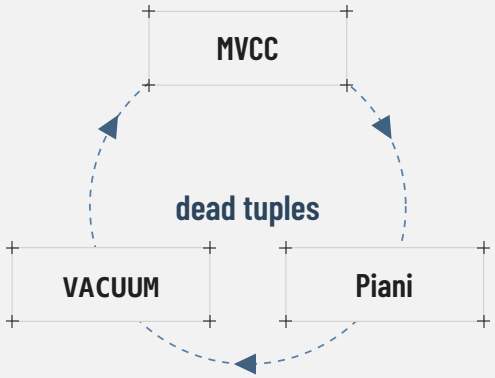




- xid corrente
- 200 milioni: oltre questa età il `VACUUM` congela per forza
- metà "passata": ~2,1 miliardi di xid
- al limite le righe non congelate sembrerebbero "nel futuro": PG blocca le scritture

Recap della giornata

MATTINA	POMERIGGIO
relazionale, SQL, `NULL`	indici B-Tree: operatore e selettività
architettura, WAL	`EXPLAIN`: stime contro realtà
MVCC, isolamento, retry	`pg_stat_statements`, JSONB, GIN/BRIN



+ Concorrenza, piani e manutenzione sono lo stesso problema: l'MVCC crea i dead tuples, il planner li incontra, il `VACUUM` li ripulisce — se nessuna transazione lunga lo blocca.

Cosa porti a casa; Giorno 2

- Materiali: script SQL, Docker Compose, dispensa e soluzioni dei laboratori, cheat-sheet.
- Conserva l'ambiente Docker: domani il confronto finale riprende alcuni esempi.
- **Giorno 2 — MongoDB**: documenti e BSON; modellazione (*embedding* contro *referencing*); Aggregation Framework; indici ed `explain()`; architettura e replica set, *write/read concern*; transazioni; confronto PostgreSQL / MongoDB.

Giorno 1 · PostgreSQL → **Giorno 2 · MongoDB**

04

CICLO 4 · PRATICA

Laboratorio

Fase 4 · Marketing e manutenzione

F4 LABORATORIO · CICLO 4

Fase 4 · Marketing e manutenzione

Il marketing chiede **tag e attributi** sugli ordini; il database cresce più del previsto. Compito: far rispondere il sito alle richieste del marketing e **tenere in salute** le tabelle.

Marketing

01 Interroga `tags` e `attrs`: contenimento, esistenza, estrazione.

02 **Ordini VIP** (tag raro) e **promo** (tag frequente): GIN su `tags`; perché `= ANY` non usa l'indice.

03 **Cerca per chiave**: `jsonb_path_ops` contro `jsonb_ops`.

04 **Report per canale**: stima sbagliata su `attrs->>'channel'` → indice su espressione.

05 **Report giornaliero**: BRIN contro B-Tree su `created_at`.

Manutenzione

06 **La tabella gonfia**: `UPDATE` di massa, `VACUUM`, `VACUUM FULL`, dimensioni.

07 **La transazione dimenticata**: un'altra sessione impedisce al `VACUUM` di pulire; trova il colpevole.

Extra: `jsonb_set`, `||`, `-` in transazione; colonna generata; soglie di autovacuum per tabella; HOT update con `fillfactor`.

Testo completo e risultati attesi: **dispensa di laboratorio, Fase 4**.